

DueFlow

DueFlow is a local-first DDL and schedule assistant with a desktop pet interface. It turns screenshots, files, notifications, pasted text and webhook payloads into an Inbox, extracts deadline tasks with an LLM-compatible provider, asks the user to confirm drafts, then generates plans, risk checks, reminders and exports.

The desktop work is inspired by OpenPets as an architecture reference only. DueFlow does not depend on OpenPets, does not copy OpenPets code or assets, and keeps its own Tauri/Python/React implementation.

What It Does

- Collects text, Markdown, PDF, image/OCR-ready files, local Inbox files and webhook payloads into a unified Inbox.
- Extracts DDL tasks, deadlines, deliverables, submit methods, missing information and source quotes.
- Requires confirmation before model output becomes committed tasks.
- Generates reverse plans, risk checks, Markdown exports and calendar `.ics` files.
- Ships a Tauri desktop shell with a main workbench and transparent always-on-top pet overlay.
- Lets the pet accept file drops, show status changes, open the main workbench, surface warnings and guide the user back to the right Inbox item.
- Supports local manifest-based desktop pet appearances and local read-only skill manifests with validation and permission boundaries.
- Provides database backup/restore, diagnostics export, self-checks and reproducible release gates.

Current Status

DueFlow is ready for local development and review as an open-source desktop project. macOS packaging is implemented as a zipped `.app` bundle. Developer ID signing, notarization and DMG distribution are documented but require project-owned Apple credentials.

The default LLM provider is `mock`, so the project can be tested without an API key. Real OpenAI-compatible providers can be configured through `.env`.

Architecture

```text

Inputs

- > Desktop API / Webhook / Streamlit / Desktop Pet
- > Inbox + deduplication
- > parser and optional OCR
- > LLM extraction
- > user confirmation
- > SQLite tasks/plans/risks
- > reminders, exports, diagnostics and desktop pet state

```

Main components:

- `api/desktop.py`: local FastAPI desktop API used by the Tauri frontend.
- `api/webhook.py`: smaller webhook API for external payloads.

- ``src/``: core database, parsing, extraction, planning, risk, export and pet-state logic.
- ``desktop/``: Tauri 2 + React desktop shell.
- ``desktop/src/PetOverlay.tsx``: desktop pet overlay.
- ``desktop/src/petRuntime.ts``: event-driven pet state and action runtime.
- ``desktop/src/skillRegistry.ts``: local skill manifest validation.
- ``desktop/src/petManifest.ts``: local desktop pet appearance validation and runtime resolution.
- ``docs/``: desktop API, release process, product plan and OpenPets reference notes.

Requirements

- Python 3.11
- Node.js 20 or newer
- Rust stable toolchain
- Conda is recommended for the Python environment
- macOS for local ``app`` packaging

Linux CI or local Rust checks for Tauri may need GTK/WebKit dependencies. See ``github/workflows/test.yml`` for the current package list.

Quick Start

Create the Python environment:

```
```bash
conda env create -f environment.yml
conda activate dueflow
```
```

Or install with pip:

```
```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```
```

For editable Python package metadata:

```
```bash
python -m pip install -e "[dev]"
```
```

Create local configuration:

```
```bash
cp .env.example .env
```
```

The default ``env.example`` uses:

```
```env
LLM_PROVIDER=mock
DATABASE_PATH=dueflow.db
```
```

```
INBOX_PATH=inbox
EXPORT_PATH=exports
'''
```

Image and screenshot intake requires OCR. On macOS, DueFlow can use the local Vision framework. On other systems, or when overriding the default, set `DUEFLOW\_OCR\_COMMAND`, for example:

```
'''env
DUEFLOW_OCR_COMMAND=tesseract {path} stdout -l chi_sim+eng
'''
```

Run the core tests:

```
'''bash
python -m pytest
'''
```

Run the command-line demo:

```
'''bash
python scripts/run_demo.py
'''
```

Desktop Development

Install frontend dependencies:

```
'''bash
cd desktop
npm install
'''
```

Run the local desktop API:

```
'''bash
conda run -n dueflow python -m uvicorn api.desktop:app --host 127.0.0.1 --port 8000
'''
```

Run the frontend in browser development mode:

```
'''bash
cd desktop
npm run dev
'''
```

Run the full Tauri shell:

```
'''bash
cd desktop
npm run tauri:dev
'''
```

The shell opens:

- `main`: the full DDL workbench.
- `pet`: a transparent always-on-top desktop pet window rendered from `index.html?view=pet`.`

The Tauri shell can also autostart the local API. Runtime paths are stored in the platform app data directory instead of the repository.

Verification

Run these before submitting a pull request:

```
```bash
python -m pytest

cd desktop
npm run build
npm run test:runtime
npm run test:smoke
npm run test:preflight
npm run test:release-manifest

cd src-tauri
cargo fmt --check
cargo check
cargo test
```
```

What the gates cover:

- `pytest`: Python pipeline, database, desktop API, packaging metadata and webhook behavior.
- `npm run build`: TypeScript and Vite production build.
- `npm run test:runtime`: pet runtime, event bridge, reminders, local skills and pet manifest logic.
- `npm run test:smoke`: isolated desktop API smoke test with temporary data; validates file intake, duplicate detection, confirmation, exports, backup/restore, diagnostics privacy and Tauri window config.
- `cargo test`: Tauri shell helper logic, including local pet import source confinement, asset copying and rollback.

Release

For local macOS packaging:

```
```bash
cd desktop
npm run release:mac
```
```

The release script runs preflight checks, builds the Tauri app, verifies the `.app` bundle, then writes:

- `desktop/release/DueFlow-Desktop\_<version>\_<arch>\_<timestamp>.app.zip`
- `desktop/release/\*.sha256`
- `desktop/release/\*.manifest.json`

See docs/desktop_release.md for the full release gate, signing and notarization notes.
Use docs/release_checklist.md before publishing a GitHub release or handing off a build.

Local Data And Privacy

DueFlow is local-first:

- SQLite database, Inbox files, exports, backups and diagnostics stay on the user's machine.
- Real model providers are optional and configured by the user.
- Diagnostics intentionally omit raw Inbox text, task titles, source quotes and extracted descriptions.
- ``.env``, databases, local Inbox files and release artifacts are ignored by Git.

See PRIVACY.md for model-provider, OCR, diagnostics and local asset privacy boundaries.

OpenPets Reference Boundary

OpenPets is used only as a reference sample for:

- event-driven interaction;
- derived pet state;
- action registration;
- plugin/manifest isolation;
- separation between pet window, control surface and desktop shell.

DueFlow does not import OpenPets packages, copy its source, reuse its assets, or adopt its Electron runtime. See docs/openpets_reference_notes.md.

Documentation

- [Desktop API](docs/desktop_api.md)
- [Desktop release](docs/desktop_release.md)
- [Release checklist](docs/release_checklist.md)
- [Changelog](CHANGELOG.md)
- [Roadmap](ROADMAP.md)
- [Contributing](CONTRIBUTING.md)
- [Code of conduct](CODE_OF_CONDUCT.md)
- [Support](SUPPORT.md)
- [Privacy](PRIVACY.md)
- [Security policy](SECURITY.md)
- [Product plan](docs/desktop_pet_product_plan.md)
- [Design notes](docs/design.md)
- [Demo notes](docs/demo.md)
- [OpenPets reference notes](docs/openpets_reference_notes.md)

Contributing

Contributions are welcome. Start with CONTRIBUTING.md, run the verification gates above, and avoid committing private data or generated local

artifacts.

Support

Use SUPPORT.md for bug reports, feature requests and support boundaries.

Code of Conduct

Participation in DueFlow project spaces follows
CODE_OF_CONDUCT.md.

Security

Please report vulnerabilities privately. See SECURITY.md.

License

DueFlow is released under the MIT License. See LICENSE.